

PASADHI/MEASUREDSYSTEMS

PASADHI: 50-100 Docker nodes with multiple coordinators and worker processes

Single-host HTTP compute-plane benchmark
A reproducible experimental extension to the PASADHI prototype

108 measured runs / 6 figures / raw request records and source fingerprints

27 October 2026 | PASADHI contributors | Experimental report v1.0

Measured GEMM execution. No external model calls. No claim of 100 physical machines, distributed LLM inference or production readiness.

Abstract

We implement and measure an experimental HTTP compute plane around PASADHI's existing float32 matrix kernel. The study varies total Docker nodes (50 or 100), coordinators (1, 2 or 4), worker processes per worker node (1 or 2), and closed-loop client concurrency (8, 32 or 64). Three repeated five-second arrival windows produce 108 runs and 99,685 requests. Every successful request is checked against a deterministic reference checksum. The complete workload recorded 0 errors, including transport failures or checksum mismatches.

At concurrency 32, the highest measured mean throughput is 214.9 requests/s for 50N / 4K / 1P and 180.7 requests/s for 100N / 4K / 1P. The best aggregate across all tested loads is 214.9 requests/s (50N / 4K / 1P, concurrency 32). These are empirical maxima over this grid, not independently validated optimal settings.

All containers share one Docker Desktop Linux engine with eight logical CPUs and approximately 7.71 GiB of memory. Adding nodes partitions the same hardware rather than adding compute capacity. Results characterize orchestration, process contention, HTTP overhead and this small GEMM workload on one host; they do not estimate a physical fleet or LLM token throughput.

1. Scope and contribution

The existing app exposes hardware settings and a text model router; it does not already provide this multi-coordinator compute plane. This report adds a separate benchmark implementation, invokes core/matrix.py::gemm_f32 on workers, and keeps its evidence separate from historical SUAY measurements and modelled fleet capacity in the reference paper.

At concurrency 32	50 nodes	100 nodes
Best configuration	50N / 4K / 1P	100N / 4K / 1P
Throughput mean +/- SD	214.9 +/- 8.2	180.7 +/- 26.9
Mean of run p95 (ms)	242.2	301.0
Measured run count	54	54

2. Experimental design and implementation

Dimension	Values / controls
Total nodes N	50, 100; includes coordinator containers and worker containers
Coordinators K	1, 2, 4; independent processes and disjoint worker partitions
Worker processes P	1 or 2 per worker container; each listens on its own port
Compute endpoints	(N - K) x P; one kernel slot per process
Load	8, 32, 64 concurrent clients; closed loop, no think time
Replication	3 runs per cell; 5-second submission window plus completion drain
Work per request	Eight float32 GEMMs, 256 x 256; 268,435,456 nominal FLOPs
BLAS threads	OPENBLAS_NUM_THREADS=OMP_NUM_THREADS=MKL_NUM_THREADS=1
Limits	80 MiB / fleet container, 256 MiB / load container; no per-node CPU quota

The external load container is excluded from N but runs on the same engine. Clients select coordinators round-robin. Each coordinator assigns a request to a least-inflight worker endpoint within its static partition, with rotating tie breaks and a semaphore allowing one outstanding call per endpoint. Each worker process has one kernel slot. There is no shared consensus store, leader election, cross-coordinator work stealing, failure retry or dynamic membership.

Requests carry a seed, matrix size and repetition count. Workers generate deterministic matrices, call the existing GEMM eight times, and return the final matrix checksum plus timings. Matrix inputs and outputs are not transferred over HTTP. This favors compute-local jobs and does not test tensor-shard communication or activation transport.

All worker processes are warmed directly before each timed run. The load order rotates by repetition (8/32/64, then 32/64/8, then 64/8/32). Node/coordinator/process configurations execute in fixed order. No API keys or project .env files are mounted into containers. Only the benchmark source, core kernel directory and results directory are mounted.

3. Throughput under concurrent load

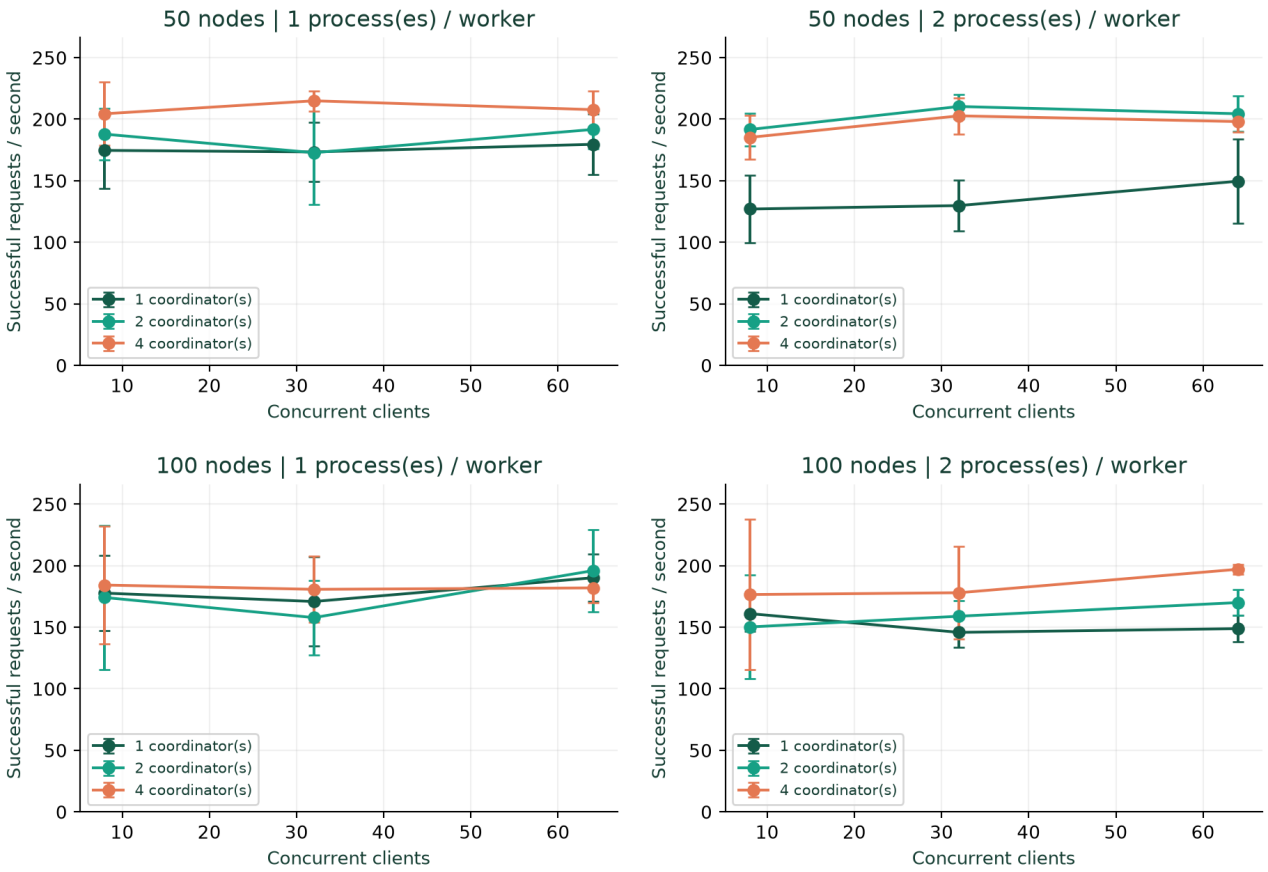


Figure 1. Successful requests per wall-clock second. Points are the mean of three runs; error bars are sample standard deviations. Each panel fixes N and P; color identifies K. N includes the coordinator containers.

Throughput counts only checksum-valid successes and divides by the full measured elapsed time, including the final in-flight request drain. A five-second arrival window can therefore have a longer denominator. This avoids counting late completions as if they finished inside the submission window.

The highest measured aggregate, 214.9 requests/s, corresponds to 57.7 nominal delivered GFLOP/s. The FLOP count includes only the eight GEMMs per request; it excludes random generation, checksums, HTTP processing and unsuccessful work. It is not an isolated kernel FLOP rate.

Comparisons should be read with the variability bars. Increasing containers on this fixed host cannot be interpreted as adding physical compute. The same CPUs also execute the load generator, Docker networking, Python handlers and coordinators.

4. End-to-end tail latency

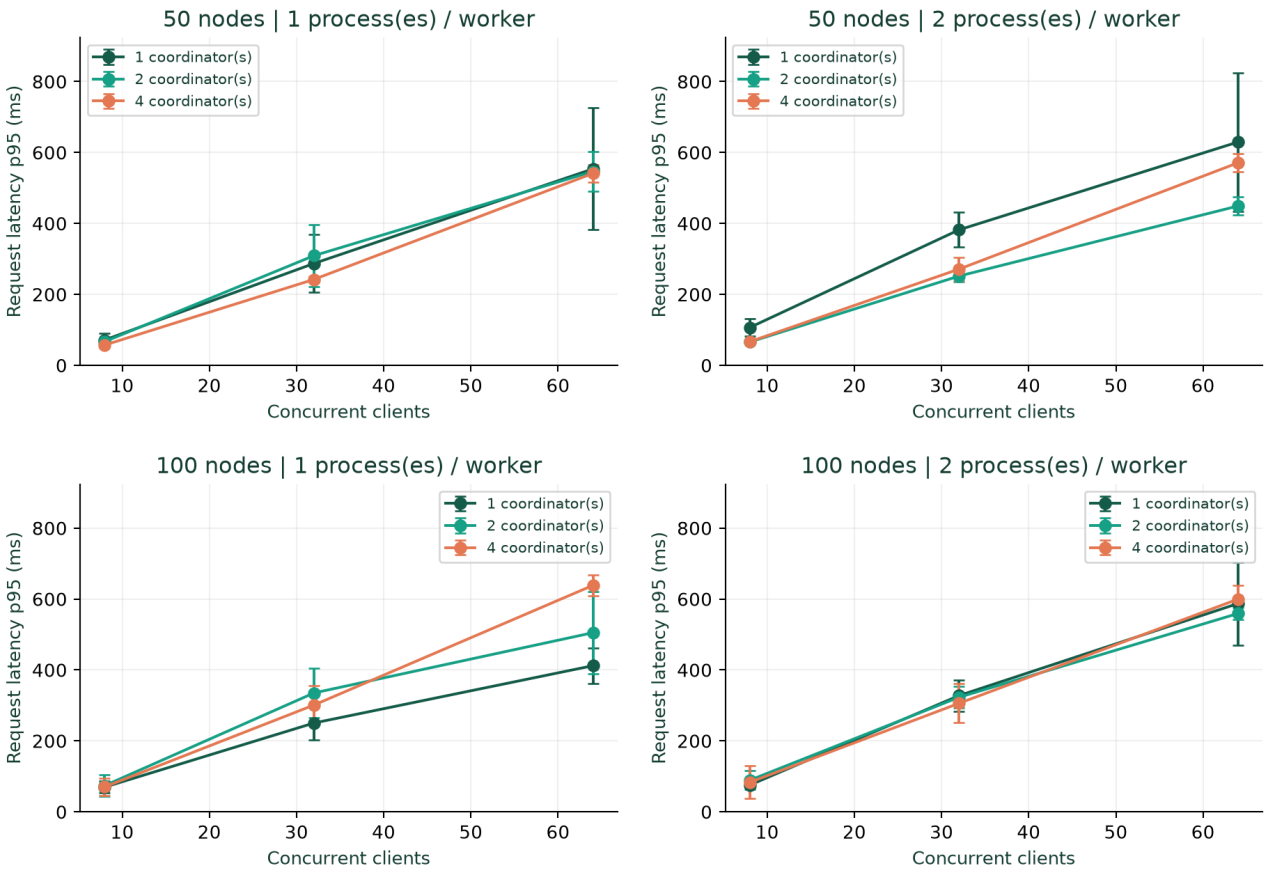


Figure 2. Mean of the per-run p95 latency estimates (three runs). Error bars are the sample standard deviation of those p95 values, not confidence intervals or per-request latency spread.

Each request latency starts immediately before its client HTTP call and ends after response decoding or an error. It includes client/engine scheduling, connection establishment, coordinator handling, worker placement/queueing, matrix generation, GEMM execution, checksum generation and response transport. It excludes the pre-run warmup.

Latency percentiles use the nearest-rank definition. The main plots average per-run quantiles; they are not quantiles of a concatenated sample. Error rows remain in the raw data and throughput denominator, but successful-request latency figures exclude errors. Error counts must therefore be read alongside latency.

A larger concurrency can improve utilization while lengthening response time. Since the generator is closed loop, it cannot quantify open-loop overload, admission stability or production arrival bursts. Small explicit semaphore queues do not imply small total waiting: operating-system and Python scheduling time can appear outside those timers.

5. Multi-coordinator and multi-process comparison

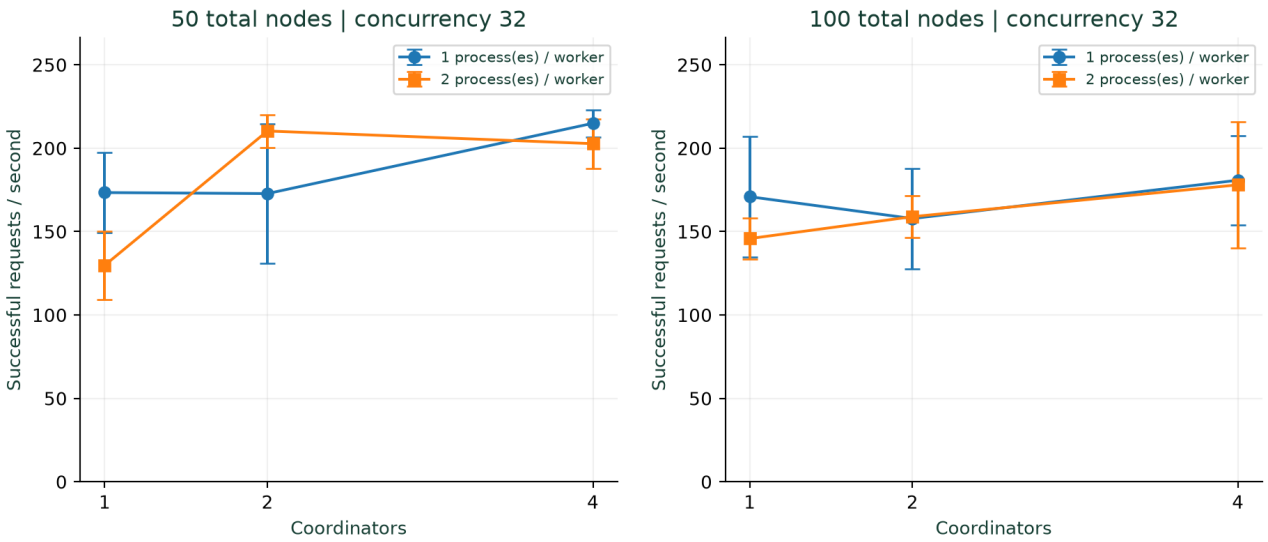


Figure 3. Coordinator scaling at concurrency 32. Each line holds processes per worker container constant. The number of worker containers is N - K, so increasing K also slightly reduces worker-node count.

N / K / P	Requests/s mean +/- SD	p95 ms	Errors
50N / 1K / 1P	173.3 +/- 23.9	287.2	0
50N / 1K / 2P	129.8 +/- 20.5	381.9	0
50N / 2K / 1P	172.7 +/- 42.0	309.1	0
50N / 2K / 2P	210.3 +/- 9.9	251.6	0
50N / 4K / 1P	214.9 +/- 8.2	242.2	0
50N / 4K / 2P	202.7 +/- 14.7	271.1	0
100N / 1K / 1P	170.9 +/- 36.2	250.3	0
100N / 1K / 2P	145.8 +/- 12.3	327.6	0
100N / 2K / 1P	157.8 +/- 30.2	335.1	0
100N / 2K / 2P	158.9 +/- 12.4	322.9	0
100N / 4K / 1P	180.7 +/- 26.9	301.0	0
100N / 4K / 2P	178.0 +/- 37.7	305.8	0

Two worker processes means two independent compute endpoints, not two machines. Coordinators have static partitions, so this experiment tests decentralized dispatch capacity but does not establish coordinator high availability.

6. Request-time decomposition

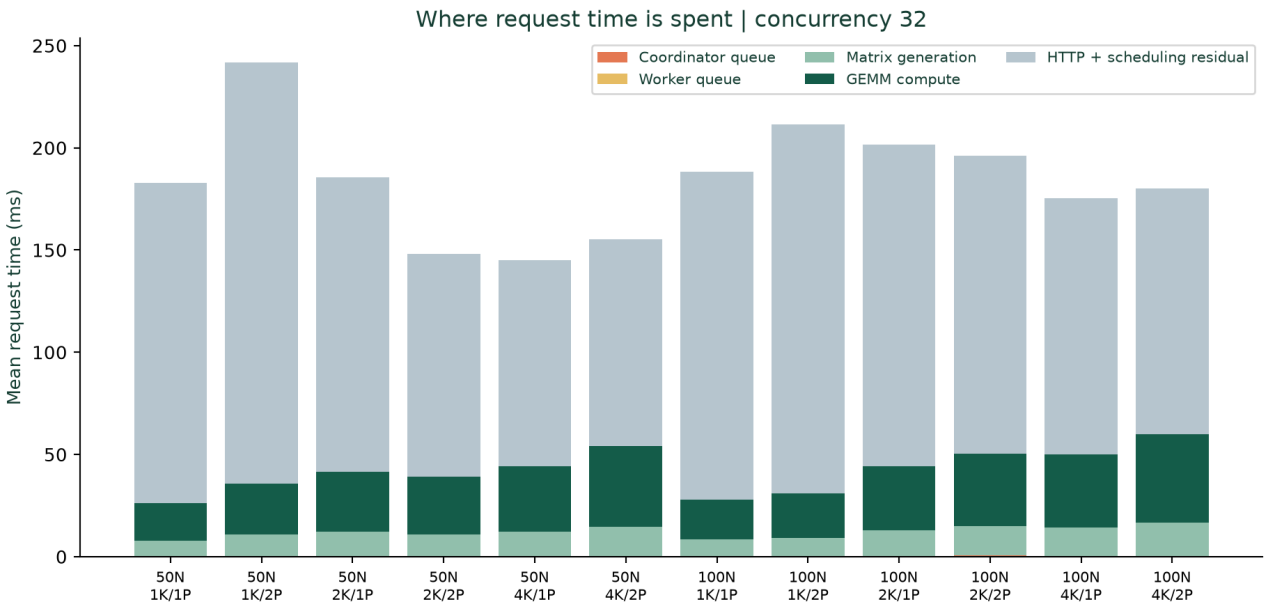


Figure 4. Mean latency components at concurrency 32, averaged over runs. The residual is client-observed mean latency minus the four instrumented intervals; it includes HTTP, checksums, serialization and uninstrumented scheduling. It must not be labelled pure network latency.

The coordinator timer begins after receiving the request body. Coordinator queue time measures waiting for the assigned endpoint semaphore. Worker queue time measures waiting for a process-local kernel slot. Matrix generation has its own timer; compute time brackets only the eight GEMM calls. All durations are taken with a local monotonic clock, so no cross-container clock synchronization is needed for these interval comparisons.

A coordinator chooses the least-inflight endpoint under a lock. Where sufficient idle endpoints exist, explicit queue times can remain low while overall latency is high. Such a result points toward shared CPU scheduling, interpreter or HTTP overhead; this experiment does not isolate which one dominates. Profiling and an independent load host are needed before attributing the residual to a particular bottleneck.

Generation and GEMM timers also include descheduling while inside their measured intervals. Thus a longer reported compute interval is wall time, not proof that each GEMM consumed proportionally more CPU cycles.

7. Resource footprint and container health

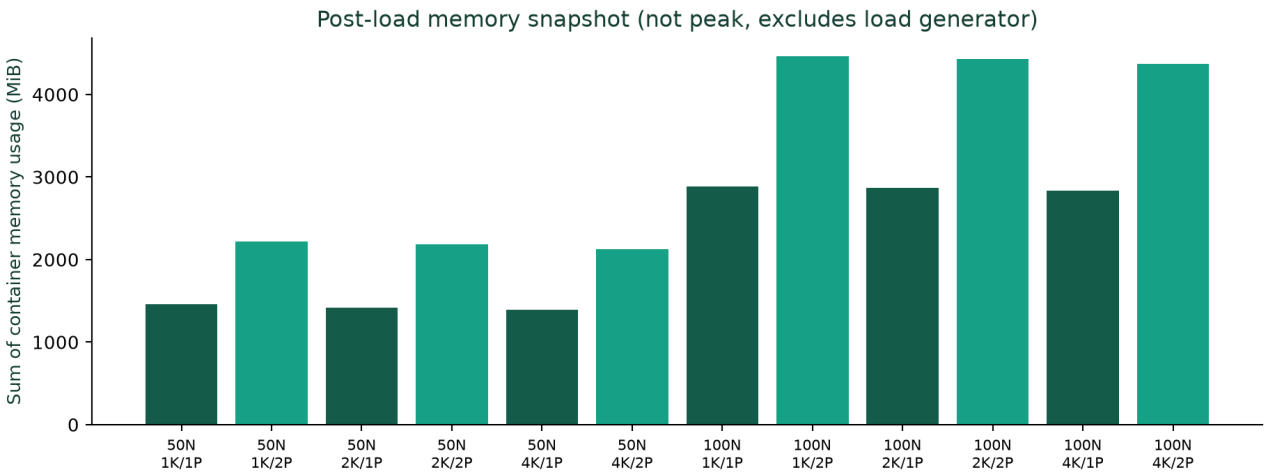


Figure 5. Sum of docker stats memory usage after each configuration finishes its load runs. The load generator has exited and is excluded. These snapshots are neither peak memory nor a time-series measurement.

The largest measured post-load fleet footprint is 4,463.3 MiB (100N / 1K / 2P). Docker Desktop engine memory, host-side processes, shared caches and the transient load container are not included in this sum.

Post-configuration inspections cover 12 configurations. They recorded 0 OOM-killed containers. Each completed configuration requires all N fleet containers to remain running before teardown. Container inspect records and Docker stats JSON lines are retained alongside request logs. A final snapshot cannot prove that there was no transient resource pressure during the run.

No coordinator-failure or network-partition experiment is included. Static worker partitions have no takeover protocol. The absence of observed request failures under this short workload must not be reported as a fault-tolerance guarantee.

8. Correctness and latency distributions

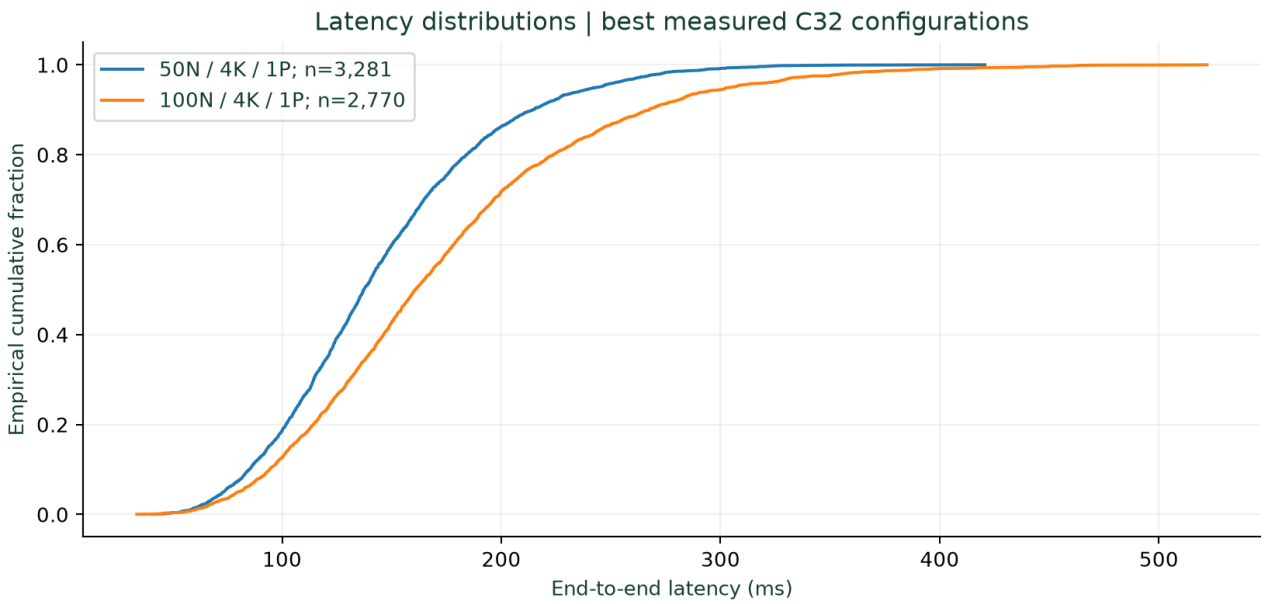


Figure 6. Empirical CDF of successful request latency for the best measured mean-throughput configuration at concurrency 32 for each node count. Requests from its three runs are pooled. Configuration selection and plotting use the same data; this is descriptive, not a held-out comparison.

Across the complete experiment, 99,685 requests were attempted and 0 were recorded as errors. A request succeeds only when its returned checksum is close to the load generator's reference (relative tolerance 1e-5, absolute tolerance 0.01). Seeds cycle through eight deterministic matrix pairs. The reference uses NumPy float32 GEMM and a float64 sum.

This verifies a useful end-to-end numerical invariant and transport wiring, but a scalar checksum cannot establish elementwise correctness for every possible input. The worker and reference also share the same numerical stack. Broader kernels, independent implementations, adversarial inputs and full matrix comparisons are outside this run.

Warmup addresses all configured worker processes. Measured worker and process identities are recorded per request, so utilization coverage can be inspected independently from container startup. No sample is silently replaced by a stub.

9. Interpretation and limits

Shared-host scaling

Fifty and one hundred nodes are container counts on the same eight-CPU engine. This is a control-plane and contention experiment, not a scale-out speedup experiment. Its curves cannot be extrapolated to a hundred networked office PCs.

Experimental compute plane

The new benchmark server invokes PASADHI's existing GEMM kernel, but it is separate from app/service.py and its Meta text router. Its results do not establish that the installed app implements distributed inference or a production scheduler.

Workload boundaries

Every task repeats a 256-square GEMM eight times. Matrices are created on the worker and only scalar checksums return. There are no model weights, tokens, storage reads, WAN links or large payload transfers. No Meta API call is made in these fleet runs.

Measurement uncertainty

Three short runs estimate variability but are insufficient for strong significance claims. Configuration order is fixed; engine warmup, background host activity, thermal effects and cache state may confound comparisons. Repeated GEMMs on identical inputs benefit from cache reuse.

Load-generator effects

The threaded Python client shares the host and has its own interpreter, socket and CPU costs. It may impose an upper throughput ceiling. Use an external load host and persistent connections in a follow-up before diagnosing coordinator limits.

Readiness boundaries

No authentication, admission quota, leader election, durable queue, retry deduplication, rolling upgrade or chaos testing is implemented here. The benchmark is bound to an isolated Compose network and publishes no host ports.

10. Reproducibility and evidence index

Run commands from the PASADHI repository root. Docker Desktop must be running. The retained image ID and source SHA-256 values in the manifest identify the actual inputs. The image is pre-existing; no external dependency install occurs during the measurements.

python bench/docker_fleet/run.py --nodes 50 100 --coordinators 1 2 4 --processes 1 2 --concurrency 8 32 64 --repeats 3 --duration 5

python bench/docker_fleet/analyze.py

[ReportLab-enabled Python] bench/docker_fleet/paper.py

Artifact	Purpose
bench/docker_fleet/node.py	Worker execution, static coordinator placement and timing
bench/docker_fleet/client.py	Warmup, closed-loop load, checksums and raw request rows
bench/docker_fleet/run.py	Compose generation, environment capture and scoped teardown
bench/results/docker_fleet/manifest.json	Engine resources, image identity, parameters, source hashes
n*_c*_r*.json / .jsonl	Per-run summary and every attempted request
n*_containers.json / _health.json / _stats.jsonl	Startup inspection, final health and memory snapshot
analysis.json / summary.csv	Run-level aggregation used by every figure and table
paper/figures/docker_fleet/	Six numbered PNG figures and SVG originals

Environment

Docker 29.8.2; Docker Desktop; kernel 6.18.54.1-microsoft-standard-WSL2; 8 logical CPUs; 7.71 GiB engine memory. Host: Windows-11-10.0.26300-SP0.

Image: sha256:661ddcc7add6c99c50061276c78894cb84dd96836119e252bc1b5d467f279e10. Started UTC: 2026-10-26T21:06:54.661722+00:00. Finished UTC: 2026-10-26T21:46:45.976399+00:00.

Reference and provenance

The layout follows paper/PASADHI-full-raw.pdf (29 pages): white background, green section headings, numbered figures and evidence-backed tables. Historical results from that volume are not reused as observations in this report. PASADHI retains its MIT license and SUAY provenance notice. The report and its source are generated from the new fleet dataset.